Activity 1.2: A Network File System

By Jorge Arévalo, Anne Baquedano, Jon González, and Laura Rodríguez

Table of Contents

Introduction 1
What is a distributed system? 1
Definition 1
Properties 1
Transparency 2
Scalability 2
Reliability and fault tolerance 3
Consistency 3
Other properties 3
SAR/SZA file server 3
Origin and purpose 4
Client-server architecture 4
Authentication and permissions 4
Protocol and file operations 4
Restrictions and implementation improvements 4
Is SAR/SZA file server a distributed system? 4
Comparison with Dropbox and OneDrive 5
Conclusion 5

Introduction

We will now analyse the SAR/SZA file service developed for the SAR course and its evolution into a small cloud file service. The original application provides remote file operations through a TCP client-server protocol. The current implementation extends it with a multithreaded server per-user storage automatic one-way synchronization and docker deployment.

The purpose of this report is to determine which properties of distributed systems are present in the application and which ones would be required for a full cloud file service. The analysis will also cover the comparison between the project and mature services such as Dropbox and OneDrive. This distinction is important: the application is distributed because its components communicate over a network, but it does not provide the redundancy, scale or availability expected from a commercial cloud platform.

What is a distributed system?

Definition

A distributed system (DS) is a collection of independent computers or processes that communicate through a network and cooperate to provide a service. The components do not share the same memory or clock, so they coordinate by exchanging messages. Ideally, the collection appears to users as a single coherent system.

SAR/SZA satisfies this basic definition. The client and server are separate processes that communicate through TCP. The client requests operations on files stored by the server. They can run on different machines or in separate Docker containers.

Properties

Transparency

Of identification

The client uses file names without needing to know how the server stores the data. This transparency is only partial because the user must still specify the server, port and local folder.

Of the physical location of the resources

The user does not know the physical disk location of a file but does know the server address and port. Files are stored under files/<user>/, so location transparency is limited.

Of replication

There is no replication. Each file has one copy on one server, so a server or disk failure makes the file unavailable.

Of parallelism

The multithreaded server can attend several clients at the same time. However, there is no locking, conflict resolution or version history.

Of sharing

Several clients can use the server, but sharing is basic. There are no sharing links, groups, configurable folder permissions or collaborative editing features.

Scalability

The server scales better than a sequential implementation because it creates a thread for each client session. Docker also makes deployment reproducible. However, the architecture remains limited to one server process and one storage location. It has no load balancing, partitioning, replication or horizontal scaling. As the number of clients and files grows, the single server and its disk become bottlenecks.

Namespaces

The current namespace is simple. Files are identified by a name inside the authenticated user's directory. The implementation separates users into directories, but the protocol does not provide a general hierarchical namespace with subdirectories, metadata, global identifiers or search.

Complexity/performance

The protocol is simple: control messages are ASCII lines ending in CRLF, while file contents are transferred as raw bytes. The synchronization worker groups repeated events and waits for files to become stable before uploading them. Uploads are written to a temporary file and then replaced atomically.

Performance is limited. Complete files are read into memory, there is no compression, and all traffic passes through one server. The line reader also receives one byte at a time, which is not optimal for high-throughput workloads.

Reliability and fault tolerance

Availability

Availability depends on the single server. If the server or its disk fails, clients cannot access the stored files. There is no failover server nor replicated storage.

Fault tolerance

The synchronization worker has limited fault tolerance for temporary network failures. It closes and reopens its connection, requeueing the operation. This queue is only in memory, so changes can be lost if the client exits unexpectedly.

Consistency

The system provides limited eventual consistency for changes made in the client folder. Creations, modifications, renames and deletions are eventually sent to the server. There is no bidirectional synchronization, conflict resolution, locking or versioning. If two clients update the same file, the later upload may overwrite the earlier one.

Other properties

Performance

The manual client and synchronization worker use separate TCP connections. This prevents automatic synchronization from blocking the interactive menu. Server threads also allow multiple sessions to progress concurrently.

Security and privacy

The application authenticates users, restricts anonymous users to read operations, separates users into directories and validates paths to prevent directory traversal outside the storage root. Uploads are performed atomically, which prevents clients from observing partially written files.

Security remains limited. Credentials are hardcoded, traffic is plain TCP without TLS, and there is no overall strong control.

Ease of use

The menu supports common file operations, and automatic synchronization avoids manual uploads for every local change. Nevertheless, the interface is terminal-based, synchronization is one-way and monitoring is not recursive.

SAR/SZA file server

Origin and purpose

The application is based on the SAR course protocol specification. It provides a simple TCP service for remote file operations. The current repository keeps the original protocol and adds automatic synchronization, multithreaded execution and containerized deployment.

Client-server architecture

The main server is *serv_fich_multithread.py*. It listens on TCP port 6012, accepts connections and starts one thread per session. Each session uses states for identification, authentication, normal operations, downloading and uploading.

The interactive client is *cli_fich.py*. It sends the commands LIST, DOWN, DOW2, UPLO, UPL2, DELE and EXIT. The SyncWorker in *sync_client.py* uses a second connection and watchdog to detect local file changes.

Authentication and permissions

The service defines three users: anonymous, sar and sza. The anonymous user can list and download files but cannot upload or delete them. The other users can perform all file operations. Each authenticated user has a separate directory on the server.

Protocol and file operations

Control messages are ASCII strings terminated by CRLF. File contents are transferred as plain bytes. A download has two stages: DOWN obtains the file size and DOW2 confirms that the client wants the bytes. An upload also has two stages: UPLO announces the name and size, and UPL2 is followed by the bytes. The other commands are USER, PASS, LIST, DELE and EXIT.

Restrictions and implementation improvements

- The maximum upload size is 10 MB
- The server requires a 50 MB free-space margin
- The watcher monitors files directly inside the configured folder
- The server validates user paths to prevent directory traversal
- Uploads use a temporary file and atomic replacement
- Temporary network failures are retried, but the queue is not persistent

The repository also contains *serv_fich.py*, an older process-based server using fork() and a shared directory. Docker uses the newer multithreaded implementation.

Is SAR/SZA file server a distributed system?

Yes, in the basic sense. The client and server are independent networked processes that cooperate through message passing. The server provides remote file access and can serve several clients concurrently. The automatic synchronizer adds another cooperating process connected to the same service.

It is not, however, a complete cloud file service. It lacks replication, horizontal scalability, high availability, bidirectional synchronization, conflict resolution, versioning and encrypted communication.

Comparison with Dropbox and OneDrive

Property	SAR/SZA	Dropbox and OneDrive
Storage	One server and local storage	Distributed and redundant
Namespace	Per-user files; limited hierarchy	Hierarchical folders, metadata and search
Concurrent access	Multiple threads, but no conflict control	Synchronization, versioning and conflict management
Location transparency	Limited; server address only known by the client	Hidden physical location
Replication	Not available	Multiple copies

Scalability	One-server scaling	Horizontal cloud scaling
Availability	Depends on one server	High availability design
Synchronization	One-way client to server	Bidirectional, multi-device
Security	Fixed users; plain TCP without TLS	Encryption and identity management
Versioning and recovery	Not available	File history and recovery
Collaboration	Not available	Sharing and collaboration

Conclusion

SAR/SZA demonstrates the main foundations of a distributed file service: independent networked components, TCP communication, remote file access, authentication and concurrent client sessions. The current implementation also provides automatic one-way synchronization, separate user directories, path validation, retry handling, atomic uploads and Docker deployment.

Its limitations are equally clear. There is one main server, no replication, no horizontal scaling, no conflict resolution, no version history and no encrypted transport. The synchronization queue is also not durable.

In conclusion, SAR/SZA ia a distributed client-server file-transfer and synchronization system, rather than as a production-grade cloud storage platform. It is useful for demonstrating the basic concepts that more advanced systems such as Dropbox and OneDrive extend with replication, scalability, reliability, security and collaboration features.